# GachaDesk

Multimodal Content Creation Tool on AMD Radeon GPU (ROCm)

*Track 1 Project Profile Document*

Team: xihuyisheng

Application: GachaDesk

Hardware: AMD Instinct MI308X (192GB HBM3)

## 1. Project Background

GachaDesk is a multimodal content creation tool built for the AMD Radeon GPU (ROCm) ecosystem. It integrates text-to-image, image-to-image editing, and image-to-video generation into a single Gradio-based web workbench, enabling users to produce storyboard-driven short videos end-to-end without leaving the browser.

Content creation is increasingly driven by generative AI, yet most existing tools either run on NVIDIA-only stacks, require expensive cloud APIs, or force users to switch between multiple disconnected applications for image, editing, and video tasks. GachaDesk addresses three pain points:

- Fragmented workflow: users juggle separate tools for text-to-image, image editing, and video generation.
- Hardware lock-in: most open-source pipelines are optimized only for CUDA, leaving AMD ROCm users underserved.
- High barrier to entry: deploying large diffusion models (20B+ parameters) requires deep ML engineering knowledge.

By deploying three state-of-the-art open models on AMD Instinct MI308X (192GB HBM3) and exposing them through a unified thin-client interface, GachaDesk lets anyone produce professional-quality visual content from a low-spec terminal while the heavy compute stays on the cloud GPU.

# 2. Target Users & Application Scenarios

### Target Users

- Rural teachers and educators who need vivid illustration materials for classroom content.
- Small business owners and live-commerce sellers producing promotional images and short videos at low cost.
- Tourism and local-product marketers creating scenic mood images and storyboard shorts.
- Content creators and short-video producers who batch-produce multi-shot frames.
- ROCm developers evaluating real-world multimodal inference performance on AMD MI300-series GPUs.

### Application Scenarios

GachaDesk supports a complete "storyboard script to finished video" workflow:

- Rural Teaching Aids - quickly generate illustration materials, making classes vivid and intuitive.
- Fashion Showcase - one-click generation of clothing on-body effects and promotional short videos.
- Tourism Promotion - generate scenic mood images and storyboard shorts to boost rural tourism marketing.
- Agricultural Live Commerce - create beautiful posters and teaser clips for agricultural products.
- Short Video Material Library - batch-produce multi-shot frames and quickly compose creative short videos.

### Workflow

The workbench provides 8 storyboard slots (2 rows x 4 columns). Users upload or generate a seed image, push it to storyboard #1, generate a video for each shot, and use the "continue frame" feature to automatically extract the last frame of each video as the input for the next shot - ensuring visual continuity across the storyboard. Finally, the 8 videos are merged into one long video with optional watermark.

# 3. System Architecture

GachaDesk adopts a thin-client + remote GPU service deployment architecture. The main project does not load any model itself; instead it calls three independently deployed sub-services via the Gradio API provided by ModelScope Studio.

### Main Project (Thin Client)

Location: src/app.py. Runs in any environment (local / edge / low-spec machine). Responsible for user interaction, storyboard orchestration, video merging and watermarking. Zero GPU dependency - only depends on requests, gradio, imageio, PIL.

### Three Remote Sub-Services (Heavy GPU Side)

Each deployed on AMD MI308X, exposing Gradio API + SSE streaming responses via ModelScope Studio:

- Qwen-Image-2512-AMD-MI308X - Text-to-Image generation (20B MMDiT diffusion model)
- Qwen-Image-Edit-2511-AMD-MI308X - Image-to-Image editing (background replacement, style transfer, outfit swap)
- Wan2.2-14B-I2V-LoRA-AMD-MI308X - Image-to-Video generation (14B DiT with distillation LoRA)

### API Invocation Flow

The main project encapsulates the full remote invocation flow via the ModelScopeImageClient class:

- GET /config - discover API endpoint and api_prefix
- POST /gradio_api/call/<api_name> - submit task, receive event_id
- GET /gradio_api/call/<api_name>/<event_id> - listen to SSE stream for results
- GET /gradio_api/file=... - download artifact through API gateway authentication
- POST /gradio_api/upload - upload image to be edited

Authentication uses a Bearer Token (read from MODELSCOPE_API_TOKEN in .env).

### Decoupled Compute Model

This architecture decouples compute from interaction: the actual ROCm inference compute is concentrated in the cloud MI308X sub-services, while the main project can run on any low-spec terminal. This is ideal for scenarios with limited terminal resources such as rural teaching aids and live-commerce broadcasting.

# 4. Model & Algorithm Introduction

## 4.1 Qwen-Image-2512 (Text-to-Image)

A 20B-parameter MMDiT (Multimodal Diffusion Transformer) model from Tongyi Qianwen. It takes a bilingual (Chinese/English) text prompt and produces high-fidelity images. The pipeline automatically appends quality suffixes (4K ultra-HD, cinematic composition) and supports 7 official recommended resolutions covering 1:1, 16:9, 9:16, 4:3, 3:4, 3:2, 2:3 aspect ratios. Negative prompt is supported to avoid low quality, malformed limbs, and AI artifacts.

## 4.2 Qwen-Image-Edit-2511 (Image-to-Image)

An image-editing MMDiT diffusion model supporting instruction-driven editing: background replacement, style transfer, character outfit swap, and more. The pipeline loads the lightx2v/Qwen-Image-Edit-2511-Lightning distillation LoRA, which reduces inference to just 4 steps while maintaining quality. Input images are center-cropped to the target aspect ratio to preserve the main subject without stretching.

## 4.3 Wan2.2-I2V-A14B (Image-to-Video)

A 14B-parameter image-to-video DiT from Wan-AI. Given a reference image and a text prompt, it generates a short video clip. The pipeline loads two distillation LoRA adapters (high_noise and low_noise) from lightx2v/Wan2.2-Distill-Loras, each at 50% weight, enabling 4-step ultra-fast inference. Output is fixed at 16 FPS, supporting 1280x720, 832x480, 768x432 resolutions and 81 or 121 frames.

## 4.4 Storyboard Merging Algorithm

Two merging modes are implemented locally in the thin client:

- 8-Image Merge: stitches 8 storyboard images at 16 frames each into a slideshow video (8 FPS).
- 8-Video Merge: concatenates 8 storyboard videos sequentially into one long video, preserving the FPS of the first video.

Both modes support optional watermark rendering: each frame gets a black-shadow + white-text label in the bottom-right corner, with font size auto-adapted to frame width.

## 4.5 Continue-Frame Mechanism

To ensure visual continuity across storyboard shots, a "continue frame" feature extracts the last frame of video N and feeds it as the input image for shot N+1. This creates a coherent visual chain from shot 1 through shot 8 without manual screenshotting.

# 5. AMD Radeon GPU / ROCm Adaptation

All three sub-services run on a single AMD Instinct MI308X (192GB HBM3) card, on top of ROCm 6.3.0+ and the ROCm build of PyTorch. Each sub-service adopts a different ROCm optimization combination tailored to its model characteristics.

## 5.1 Qwen-Image-2512 Optimizations

- Precision: bfloat16 (native ROCm support)
- Attention: flash_attention_2 with preferred_rocm_fa_library('aotriton') backend
- MatMul: allow_tf32=True + set_float32_matmul_precision('high') for MI300+ TF32 speedup
- VRAM: full GPU loading - 20B MMDiT entirely in 192GB HBM3, zero paging
- Fallback: VAE float16 decoding falls back to float32 on failure; torch.cuda.synchronize() at key points

## 5.2 Qwen-Image-Edit-2511 Optimizations

- Precision: bfloat16
- Attention: enable_flash_sdp(True) + enable_mem_efficient_sdp(True), math_sdp disabled
- Env vars: TORCH_ROCM_AOTRITON_ENABLE_EXPERIMENTAL=1, FLASH_ATTENTION_TRITON_AMD_ENABLE=TRUE
- Env vars: HIP_FORCE_DEV_KERNARG=1, PYTORCH_HIP_ALLOC_CONF=garbage_collection_threshold:0.8,max_split_size_mb:512
- Cache persistence: TRITON_CACHE_DIR / TORCH_EXTENSIONS_DIR redirected to /mnt/workspace to avoid kernel recompilation after restart (first compile ~20 min, subsequent instant)
- Acceleration: lightx2v Lightning distillation LoRA, 4-step inference
- Resolution lock: fixed 1664x928 to avoid shape-triggered kernel recompilation

## 5.3 Wan2.2-I2V Optimizations

- Precision: float16
- Attention: xformers (falls back to default on failure)
- Acceleration: high_noise + low_noise dual distillation LoRA adapters (50% weight each), 4-step inference
- Output: fixed 16 FPS, 1280x720 / 832x480 / 768x432, 81 or 121 frames
- Monitoring: integrates rocm-smi for real-time temperature and core/memory clock

## 5.4 Common Optimizations

- Full local inference: all models entirely loaded into MI308X VRAM, no quantization sharding or CPU offload
- Gradio + MCP: mcp_server=True, each API endpoint mapped to an MCP tool for agent invocation
- Dual-channel logging: logger.info / print output to both console and Gradio runtime log panel
- VRAM hygiene: gc.collect() + torch.cuda.empty_cache() before and after each inference

*- End of Document -*